

# 从 Mip-NeRF 360 到世界模型的工程实践指南

基于 3D\_Reconstruction\_ing 仓库

3D\_Reconstruction\_ing 项目笔记

2026 年 7 月 6 日

## 摘要

本文面向工程实践，基于 3D\_Reconstruction\_ing 仓库，给出从 Mip-NeRF 360 数据集出发，经 COLMAP 几何重建、Nerfstudio NeRF 训练、3D Gaussian Splatting (3DGS) 以及语义标注与世界模型场景状态构建的完整 Pipeline。文中所有命令与代码示例均可在仓库结构与环境配置下直接运行或微调使用，与配套的理论综述文档构成“原理 + 实践”完整一套笔记。

## 目录

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>总体说明与仓库结构</b>                           | <b>2</b> |
| <b>2</b> | <b>环境配置</b>                                | <b>3</b> |
| 2.1      | 几何实验环境（仅 COLMAP / pycolmap） . . . . .      | 3        |
| 2.2      | 完整世界重建环境（Nerfstudio + 3DGS + 语义） . . . . . | 3        |
| <b>3</b> | <b>数据组织：Mip-NeRF 360 场景布局</b>              | <b>4</b> |
| <b>4</b> | <b>COLMAP 几何重建实践</b>                       | <b>4</b> |
| 4.1      | 创建 COLMAP 工程与特征提取 . . . . .                | 4        |
| 4.2      | 特征匹配与稀疏重建 . . . . .                        | 5        |
| 4.3      | 可选：稠密 MVS 与点云导出 . . . . .                  | 5        |
| <b>5</b> | <b>Nerfstudio：NeRF 训练与渲染实践</b>             | <b>6</b> |
| 5.1      | 预处理：ns-process-data（内部调用 COLMAP） . . . . . | 6        |
| 5.2      | 训练 NeRF（nerfacto） . . . . .                | 7        |
| 5.3      | 使用 ns-render 导出视频 . . . . .                | 7        |
| <b>6</b> | <b>3DGS：使用 splatfacto 训练 3D 高斯</b>         | <b>8</b> |
| 6.1      | 训练 splatfacto . . . . .                    | 8        |

|                                 |    |
|---------------------------------|----|
| 1 总体说明与仓库结构                     | 2  |
| 7 语义 Pipeline: 从 2D 分割到 3D 语义点云 | 8  |
| 7.1 2D 语义分割与可选 3D 融合            | 9  |
| 7.2 将语义挂接到 World State          | 9  |
| 7.3 用 Python 查询语义信息             | 9  |
| 7.4 3D 语义可视化                    | 10 |
| 8 语义 NeRF 与语义 3DGS 训练与渲染        | 11 |
| 8.1 语义 NeRF: 训练                 | 11 |
| 8.2 语义 NeRF: 渲染                 | 11 |
| 8.3 语义 3DGS: 在 splatfacto 上挂语义头 | 12 |
| 9 统一静态场景状态 (Scene State)        | 12 |
| 10 端到端命令速查                      | 13 |
| 11 总结                           | 14 |

## 1 总体说明与仓库结构

本指南假定读者已克隆并切换到 3D\_Reconstruction\_ing 仓库根目录:

```
cd /home/wenhanxiao/code/3D_Reconstruction_ing
```

仓库结构 (节选) 如下:

```
3D_Reconstruction_ing/
  README.md
  env/
    worldrecon.yml # Nerfstudio + gsplat 等完整环境
  docs/
    review_world_model_theory.tex # 理论综述 (源文件)
    review_world_model_theory.md # 理论综述 (Markdown 导出)
    review_world_model_engineering.tex # 工程实践指南 (源文件)
    review_world_model_engineering.md # 工程实践指南 (Markdown 导出)
  mipnerf360/
    db/
      drjohnson/ # 示例场景 (可扩展为 playroom 等)
  scripts/
    run_semantic_labeling.py
    train_semantic_nerf.py
    train_semantic_3dgs.py
    render_semantic_nerf.py
```

```
render_semantic_3dgs.py
view_semantic_3d.py
query_semantics_example.py
src/
  world_model/
  semantic_nerf/
  semantic_3dgs/
```

后文工程实践均围绕上述结构展开。

## 2 环境配置

### 2.1 几何实验环境 (仅 COLMAP / pycolmap)

若只想做传统 SfM / MVS 与 pycolmap 实验, 可使用最小 Python 虚拟环境:

```
cd /home/wenhanxiao/code/3D_Reconstruction_ing

python -m venv .venv
source .venv/bin/activate # Windows 使用 .venv\Scripts\activate

python -m pip install --upgrade pip
pip install -r requirements.txt
```

### 2.2 完整世界重建环境 (Nerfstudio + 3DGS + 语义)

要跑通从 Mip-NeRF 360 → COLMAP → Nerfstudio → 3DGS → 语义/世界模型的整条链路, 推荐使用项目提供的 worldrecon 环境:

```
cd /home/wenhanxiao/code/3D_Reconstruction_ing

conda env create -f env/worldrecon.yml
conda activate worldrecon
```

该环境中包含:

- 基础依赖: Python、ffmpeg、cmake、ninja、git 等;
- 深度与重建栈: nerfstudio、gsplat、torch、opencv-python、open3d、trimesh 等;
- 语义相关依赖: transformers、timm、einops、accelerate、supervision 等。

建议将 COLMAP 本身单独安装（系统包或独立 Conda 环境），worldrecon 环境主要负责 Nerfstudio / NeRF / 3DGS 与语义部分。

### 3 数据组织：Mip-NeRF 360 场景布局

以官方 Mip-NeRF 360 的 drjohnson 场景为例，推荐目录结构为：

```
3D_Reconstruction_ing/  
  mipnerf360/  
    db/  
      drjohnson/  
        images/ # 原始多视角图像  
        sparse/0/ # 可选：COLMAP SfM 输出  
        dense/0/ # 可选：COLMAP 稠密输出  
        ...
```

约定若干环境变量方便后续命令书写：

```
REPO_ROOT=/home/wenhanxiao/code/3D_Reconstruction_ing  
DATA_ROOT=$REPO_ROOT/mipnerf360  
SCENE_NAME=db/drjohnson  
SCENE_ROOT=$DATA_ROOT/$SCENE_NAME
```

只要能在 \$SCENE\_ROOT/images 下看到所有图像文件，就可以继续几何或辐射场实验。

## 4 COLMAP 几何重建实践

本节给出基于命令行的 COLMAP 标准 Pipeline。

### 4.1 创建 COLMAP 工程与特征提取

```
REPO_ROOT=/home/wenhanxiao/code/3D_Reconstruction_ing  
DATA_ROOT=$REPO_ROOT/mipnerf360  
SCENE_NAME=db/drjohnson  
SCENE_ROOT=$DATA_ROOT/$SCENE_NAME  
  
PROJECT_PATH=$SCENE_ROOT/colmap_project  
IMAGE_PATH=$SCENE_ROOT/images  
mkdir -p $PROJECT_PATH
```

特征提取：

```
colmap feature_extractor \  
  --database_path $PROJECT_PATH/database.db \  
  --image_path $IMAGE_PATH \  
  --ImageReader.single_camera 0 \  
  --ImageReader.camera_model SIMPLE_RADIAL
```

## 4.2 特征匹配与稀疏重建

```
colmap exhaustive_matcher \  
  --database_path $PROJECT_PATH/database.db
```

运行 Mapper 做增量式 SfM：

```
mkdir -p $PROJECT_PATH/sparse/0  
  
colmap mapper \  
  --database_path $PROJECT_PATH/database.db \  
  --image_path $IMAGE_PATH \  
  --output_path $PROJECT_PATH/sparse
```

完成后，\$PROJECT\_PATH/sparse/0 中包含：

- cameras.bin / images.bin / points3D.bin;
- 工程配置与日志。

可进一步转为文本以便阅读和调试：

```
colmap model_converter \  
  --input_path $PROJECT_PATH/sparse/0 \  
  --output_path $PROJECT_PATH/sparse/0 \  
  --output_type TXT
```

此时，cameras.txt / images.txt / points3D.txt 中的相机位姿、3D 点云与可见关系即可被后续 Python 代码或 Nerfstudio 利用。

## 4.3 可选：稠密 MVS 与点云导出

如需高密度点云，可在同一工程下继续执行 MVS：

```
mkdir -p $PROJECT_PATH/dense  
cp -r $PROJECT_PATH/sparse/0 $PROJECT_PATH/dense/
```

```
colmap image_undistorter \  
  --image_path $IMAGE_PATH \  
  --input_path $PROJECT_PATH/dense/0 \  
  --output_path $PROJECT_PATH/dense/0_undistort \  
  --output_type COLMAP  
  
colmap patch_match_stereo \  
  --workspace_path $PROJECT_PATH/dense/0_undistort \  
  --workspace_format COLMAP  
  
colmap stereo_fusion \  
  --workspace_path $PROJECT_PATH/dense/0_undistort \  
  --workspace_format COLMAP \  
  --input_type geometric \  
  --output_path $PROJECT_PATH/dense/fused.ply
```

导出的 fused.ply 可用 MeshLab、CloudCompare 或 Open3D 等工具可视化和后处理。

## 5 Nerfstudio: NeRF 训练与渲染实践

本节以 Nerfstudio 的 nerfacto 模型为例，跑通从 COLMAP 结果到 NeRF 训练与视频渲染的 Pipeline。

### 5.1 预处理：ns-process-data（内部调用 COLMAP）

在 worldrecon 环境中执行：

```
conda activate worldrecon  
  
REPO_ROOT=/home/wenhanxiao/code/3D_Reconstruction_ing  
DATA_ROOT=$REPO_ROOT/mipnerf360  
SCENE_NAME=db/drjohnson  
SCENE_ROOT=$DATA_ROOT/$SCENE_NAME  
OUT_DIR=$SCENE_ROOT/ns_processed  
  
ns-process-data images \  
  --data $SCENE_ROOT/images \  
  --output-dir $OUT_DIR \
```

```
--matching-method exhaustive
```

该命令会自动：

- 执行特征提取、匹配与 SfM；
- 生成 Nerfstudio 所需的 images/ 与 transforms.json；
- 写入相关元数据与日志。

## 5.2 训练 NeRF (nerfacto)

```
conda activate worldrecon

ns-train nerfacto \
  --data $OUT_DIR \
  --output-dir $SCENE_ROOT/ns_runs/drjohnson_nerfacto
```

训练过程中默认会启动 Web Viewer (通常位于 <http://127.0.0.1:7007>)，可实时观察渲染结果与损失变化。

训练输出目录大致为：

```
$SCENE_ROOT/ns_runs/drjohnson_nerfacto/
  config.yml # 训练配置 (后续渲染会用到)
  nerfstudio_models/ # 模型 checkpoints
```

## 5.3 使用 ns-render 导出视频

使用默认相机轨迹渲染环绕视频：

```
RUN_DIR=$SCENE_ROOT/ns_runs/drjohnson_nerfacto

ns-render dataset \
  --load-config $RUN_DIR/config.yml \
  --output-path $RUN_DIR/render_orbit.mp4
```

使用自定义相机路径：

1. 在 Web Viewer 中录制并保存 camera\_path.json；
2. 执行：

```
RUN_DIR=$SCENE_ROOT/ns_runs/drjohnson_nerfacto
```

```
ns-render camera-path \  
  --load-config $RUN_DIR/config.yml \  
  --camera-path $RUN_DIR/camera_path.json \  
  --output-path $RUN_DIR/render_custom.mp4
```

## 6 3DGS: 使用 *splatfacto* 训练 3D 高斯

在完成 *ns-process-data* 之后, 可以使用 Nerfstudio 内置的 *splatfacto* 训练 3DGS 表示。

### 6.1 训练 *splatfacto*

```
conda activate worldrecon  
  
REPO_ROOT=/home/wenhanxiao/code/3D_Reconstruction_ing  
DATA_ROOT=$REPO_ROOT/mipnerf360  
SCENE_NAME=db/drjohnson  
SCENE_ROOT=$DATA_ROOT/$SCENE_NAME  
OUT_DIR=$SCENE_ROOT/ns_processed  
  
ns-train splatfacto \  
  --data $OUT_DIR \  
  --output-dir $SCENE_ROOT/ns_runs/drjohnson_splatfacto
```

训练完成后, 同样可以用 *ns-render* 渲染视频:

```
RUN_DIR=$SCENE_ROOT/ns_runs/drjohnson_splatfacto  
  
ns-render dataset \  
  --load-config $RUN_DIR/config.yml \  
  --output-path $RUN_DIR/render_orbit.mp4
```

同时, 3D 高斯参数可被后续语义 3DGS 模块加载和扩展。

## 7 语义 Pipeline: 从 2D 分割到 3D 语义点云

本节对应脚本目录 *scripts/* 与模块 *src/world\_model*, 给出从 2D 分割到 3D 语义场景的实际使用方法。



## 7.1 2D 语义分割与可选 3D 融合

在已有 `ns_processed` 的前提下, 可以运行:

```
conda activate worldrecon
cd /home/wenhanxiao/code/3D_Reconstruction_ing

# 仅 2D 语义
python scripts/run_semantic_labeling.py \
    --processed-dir mipnerf360/db/playroom/ns_processed \
    --output-dir mipnerf360/db/playroom/semantic \
    --scene-id mipnerf360/db/playroom

# 2D + 3D 语义融合 (需 COLMAP sparse)
python scripts/run_semantic_labeling.py \
    --processed-dir mipnerf360/db/playroom/ns_processed \
    --sparse-dir mipnerf360/db/playroom/sparse/0 \
    --output-dir mipnerf360/db/playroom/semantic \
    --scene-id mipnerf360/db/playroom
```

输出目录 (如 `mipnerf360/db/playroom/semantic/`) 包含:

- `semantic_scene.json`: 语义场景描述 (图像级元数据、可选 3D 点语义);
- `label_maps/*.npy`: 每张图的像素级类别标签。

## 7.2 将语义挂接到 World State

在场景的 world state JSON 中增加语义表示, 例如:

```
"semantics": {
  "semantic_scene_json": "semantic/semantic_scene.json"
}
```

当使用 `SceneState` 加载该场景时, 可一并加载语义层, 实现几何/辐射场/3DGS/语义的统一管理。

## 7.3 用 Python 查询语义信息

脚本 `scripts/query_semantics_example.py` 与模块 `src/world_model/semantics.py` 提供了高层查询 API。以下是一个示例片段:

```
from pathlib import Path
from src.world_model import load_scene_state
```

```

from src.world_model.semantics import load_semantic_scene

REPO_ROOT = Path("/home/wenhanxiao/code/3D_Reconstruction_ing")
state = load_scene_state(
    REPO_ROOT / "mipnerf360/db/playroom/world_state.playroom.json"
)
sem = state.load_semantic_scene()
if sem is None:
    sem = load_semantic_scene(
        REPO_ROOT / "mipnerf360/db/playroom/semantic/semantic_scene.json"
    )

# 按类别查询: 包含 "chair" 的图像与 3D 点
result = sem.query_by_class("chair", include_2d=True, include_3d=True)
print("包含 chair 的图像数:", len(result["images"]))
print("包含 chair 的 3D 点数:", len(result["points"]))

# 按 3D 包围盒查询
points_in_region = sem.query_region(
    bbox_min=(0, 0, 0), bbox_max=(2, 2, 2), class_filter="table"
)

```

通过 `query_by_class`、`query_region`、`get_semantic_at_point` 等接口，可以将语义层作为“世界记忆”的查询入口。

## 7.4 3D 语义可视化

脚本 `scripts/view_semantic_3d.py` 支持在 Open3D Viewer 中按类别上色查看 3D 语义点云：

```

conda activate worldrecon
cd /home/wenhanxiao/code/3D_Reconstruction_ing

# 方式一: 直接给 semantic_scene.json
python scripts/view_semantic_3d.py \
    --semantic-json mipnerf360/db/playroom/semantic/semantic_scene.json

# 方式二: 通过 world_state 自动定位
python scripts/view_semantic_3d.py \
    --world-state mipnerf360/db/playroom/world_state.playroom.json

```

在无显示器环境（如远程服务器）下，可导出彩色 PLY 后在本机查看：

```
python scripts/view_semantic_3d.py \  
--semantic-json mipnerf360/db/playroom/semantic/semantic_scene.json \  
--export-ply semantic_colored.ply
```

## 8 语义 NeRF 与语义 3DGS 训练与渲染

本节围绕 `src/semantic_nerf` 与 `src/semantic_3dgs`，给出训练与渲染流程。

### 8.1 语义 NeRF：训练

在已有 `ns_processed` 与 `semantic/label_maps/*.npy` 的前提下：

```
conda activate worldrecon  
cd /home/wenhanxiao/code/3D_Reconstruction_ing  
  
python scripts/train_semantic_nerf.py \  
--processed-dir mipnerf360/db/playroom/ns_processed \  
--semantic-dir mipnerf360/db/playroom/semantic \  
--output-dir mipnerf360/db/playroom/semantic_nerf_runs \  
--num-classes 150 \  
--steps 10000
```

输出为：

```
mipnerf360/db/playroom/semantic_nerf_runs/semantic_nerf.pt
```

该权重文件中包含语义 NeRF 的 MLP 参数，可被渲染脚本加载。

### 8.2 语义 NeRF：渲染

```
python scripts/render_semantic_nerf.py \  
--checkpoint mipnerf360/db/playroom/semantic_nerf_runs/semantic_nerf.pt \  
--processed-dir mipnerf360/db/playroom/ns_processed \  
--output-dir mipnerf360/db/playroom/semantic_nerf_runs/renders \  
--frame-idx 0
```

输出目录中通常包含：

- `rgb.png`：渲染 RGB；
- `semantic_colormap.png`：按类别着色的语义图；
- `rgb_semantic_sidebyside.png`：左右对比图。

### 8.3 语义 3DGS：在 splatfacto 上挂语义头

第一步，确保已经训练好 splatfacto：

```
conda activate worldrecon
cd /home/wenhanxiao/code/3D_Reconstruction_ing

ns-train splatfacto \
  --data mipnerf360/db/playroom/ns_processed \
  --output-dir mipnerf360/db/playroom/ns_runs/playroom_splatfacto \
  --max-num-iterations 30000
```

其中 ns\_runs/playroom\_splatfacto/ns\_processed/splatfacto/ 下会生成时间戳命名的子目录，内含 config.yml。

第二步，训练语义头：

```
python scripts/train_semantic_3dgs.py \
  --splatfacto-config \
    mipnerf360/db/playroom/ns_runs/playroom_splatfacto/\
ns_processed/splatfacto/YYYY-MM-DD_HHMMSS/config.yml \
  --processed-dir mipnerf360/db/playroom/ns_processed \
  --semantic-dir mipnerf360/db/playroom/semantic \
  --output-dir mipnerf360/db/playroom/semantic_3dgs_runs \
  --steps 3000
```

注意将 YYYY-MM-DD\_HHMMSS 替换为实际生成的时间戳目录名。

渲染语义 3DGS：

```
python scripts/render_semantic_3dgs.py \
  --checkpoint mipnerf360/db/playroom/semantic_3dgs_runs/semantic_3dgs.pt \
  --processed-dir mipnerf360/db/playroom/ns_processed \
  --output-dir mipnerf360/db/playroom/semantic_3dgs_runs/renders \
  --frame-idx 0
```

## 9 统一静态场景状态 (Scene State)

为了在世界模型实验中复用同一真实场景，仓库推荐为每个场景维护一个静态场景状态描述文件，例如：

```
mipnerf360/db/playroom/world_state.playroom.json
```

其内部通常包含：

- 场景标识、根目录与坐标系定义；

- `representations.colmap.*`: 几何层结果路径;
- `representations.nerfstudio.*`: NeRF / Nerfacto 运行目录与配置;
- `representations.gaussians.*`: 3DGS / splatfacto 相关路径;
- `representations.semantics.*`: 语义层 (如 `semantic_scene.json`)。

在 Python 中, 可使用 `src/world_model/scene_state.py` 中的工具函数:

```
from pathlib import Path
from src.world_model import load_scene_state

REPO_ROOT = Path("/home/wenhanxiao/code/3D_Reconstruction_ing")
state = load_scene_state(
    REPO_ROOT / "mipnerf360/db/playroom/world_state.playroom.json"
)

# 访问几何 / NeRF / 3DGS / 语义等子模块路径
print(state.scene_id, state.root)
print(state.representations.colmap)
print(state.representations.nerfstudio)
print(state.representations.gaussians)
print(state.representations.semantics)
```

通过 Scene State, 可以将 COLMAP 位姿、Nerfstudio 模型、3DGS 表示与语义场景统一挂接到一个“世界记忆单元”, 后续无论做渲染、编辑还是语义查询, 都只需先加载同一个 SceneState。

## 10 端到端命令速查

以 `db/drjohnson` 为例, 从原始图像到 NeRF 渲染与 3DGS、语义的端到端流程可简要概括为:

```
# 0. 激活环境
conda activate worldrecon

# 1. 路径约定
REPO_ROOT=/home/wenhanxiao/code/3D_Reconstruction_ing
DATA_ROOT=$REPO_ROOT/mipnerf360
SCENE_NAME=db/drjohnson
SCENE_ROOT=$DATA_ROOT/$SCENE_NAME
```

```
# 2. Nerfstudio 预处理 (内部调用 COLMAP)
OUT_DIR=$SCENE_ROOT/ns_processed
ns-process-data images \
  --data $SCENE_ROOT/images \
  --output-dir $OUT_DIR

# 3. 训练 NeRF (nerfacto)
ns-train nerfacto \
  --data $OUT_DIR \
  --output-dir $SCENE_ROOT/ns_runs/drjohnson_nerfacto

# 4. 渲染默认相机轨迹视频
RUN_DIR=$SCENE_ROOT/ns_runs/drjohnson_nerfacto
ns-render dataset \
  --load-config $RUN_DIR/config.yml \
  --output-path $RUN_DIR/render_orbit.mp4
```

在此基础上，可以选择性地：

- 使用 COLMAP 命令行深入理解几何层；
- 使用 splatfacto 训练并比较 3DGS；
- 运行 run\_semantic\_labeling.py、语义 NeRF / 3DGS 训练脚本，构建可查询的语义世界；
- 通过 Scene State 将上述内容整合为统一世界模型实验平台。

## 11 总结

本文给出了围绕 3D\_Reconstruction\_ing 仓库的工程实践指南，涵盖从数据布局、环境配置、COLMAP 几何重建、Nerfstudio NeRF 与 3DGS 训练、到语义标注与世界模型场景状态构建的完整操作路径。配合理论综述文档 review\_world\_model\_theory.tex，读者可以一边推导关键公式，一边在真实多视角数据上完成从三维重建到世界模型表征的端到端实验。